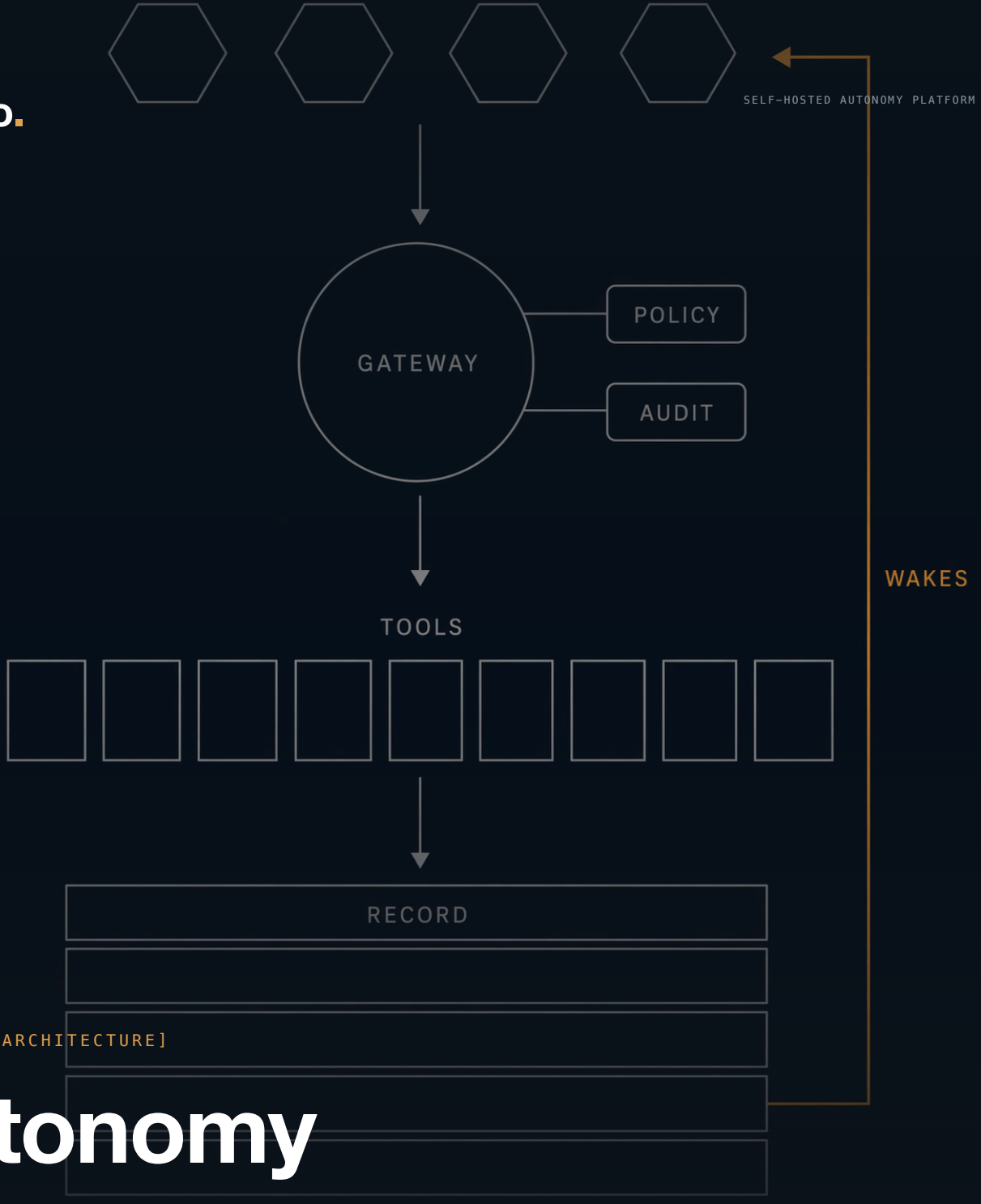
veoveo.



[PLATFORM ARCHITECTURE]

Autonomy Harness

A reactive and proactive cognition stack: autonomous agents that run live operations — on infrastructure you own.

Table of Contents

#	Section
1	**Executive Summary**
2	**Capabilities**
3	**World Model**
4	**Data Flow**
5	**Agent Kernel**
6	**Edge, Access, and the Gateway**
7	**Hosted Capabilities**
8	**Durable Tasks and Shared Planes**
9	**Recording Hub**
10	**SUMO Showcase**
11	**Operations and Deployment**
12	**Verification and Observability**
13	**Component Reference**

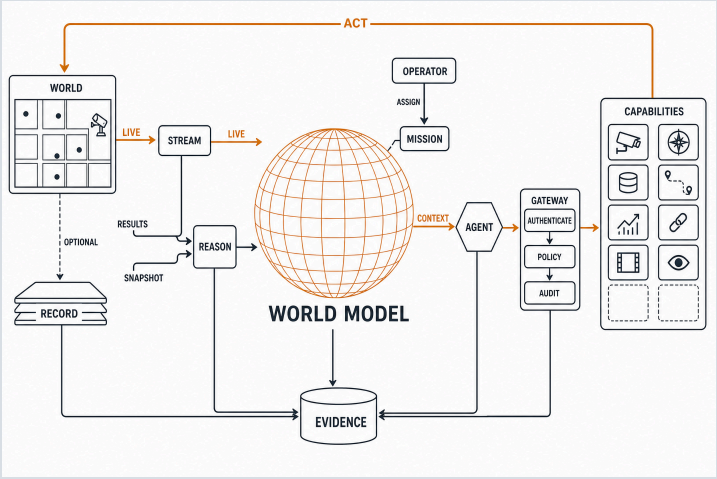
1. Executive Summary

The Autonomy Harness is a reactive and proactive cognition stack: autonomous agents that run live operations on infrastructure the operator owns. Every sensor, memory, and decision the operation collects fuses into a world model of its operational reality. Agents hunt through it around the clock — they catch conditions the moment they change, decide, and intervene — and between events they drive the mission forward on their own initiative, analyzing, planning, and tasking work under rules of engagement the operator sets.

17	Full	2.56 s
Hosted Servers	MCP Surface	Measured Archive Pass

Key Findings

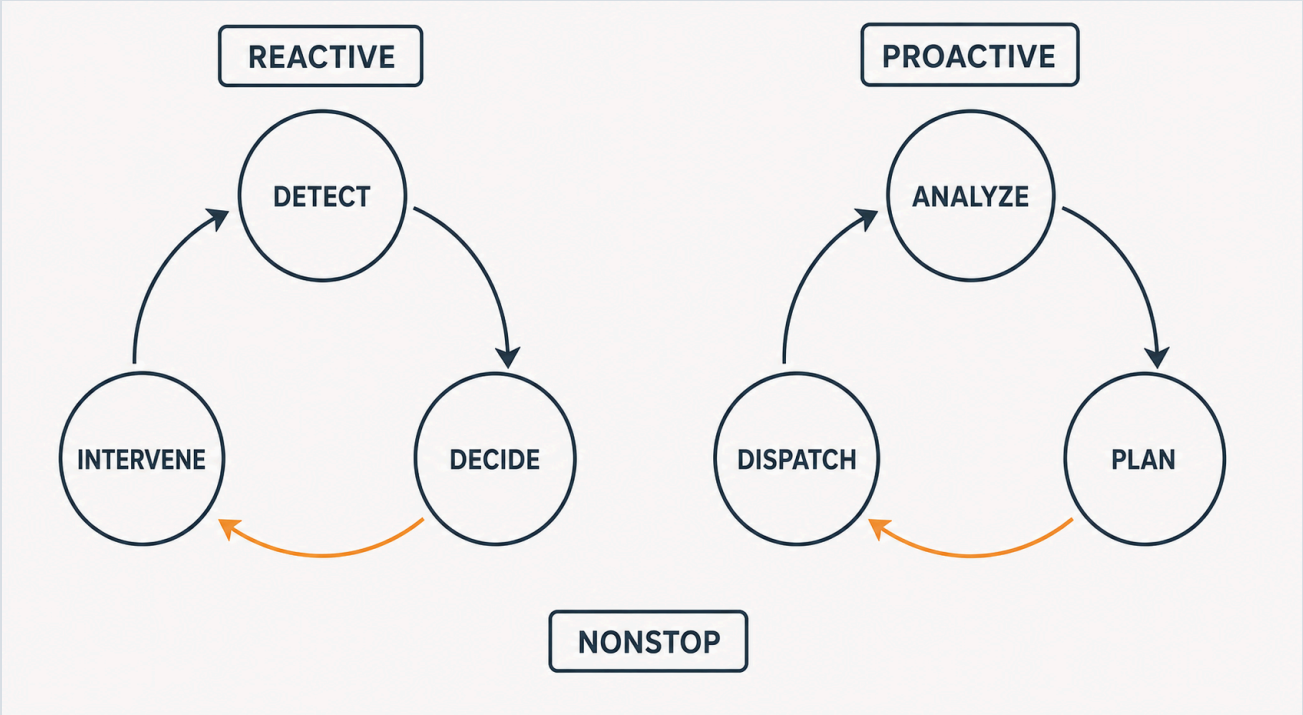
- **Cognition is governed from outside.** Every action clears one authenticated door and answers to installation policy, refused by default when in doubt. A model can be wrong, drift, or swallow hostile instructions and still cannot act beyond the envelope.
- **The world model is the ground truth and the evidence chain.** Sensors, memories, and agent decisions stream into one durable record. Any run replays, any moment is queryable, and when someone asks why the system acted, the record answers.
- **Long work carries its own guarantees.** Every long operation is a durable task with a declared recovery class; agents detach, and results wake them within a second of completion — across process death.
- **The installation is sovereign.** Built for contested and disconnected operations: one Helm workload model runs in local k3d, fielded Kubernetes, and a verified offline bundle with provenance included.
- **A live city proves the loop.** A pinned SUMO traffic world drives perceive → decide → act end to end through the same contracts as every other capability.



THE WHOLE HARNESS: THE WORLD RECORDED AND MODELED, THE MISSION IN THE MODEL, AGENTS ACTING THROUGH THE GOVERNED GATEWAY INTO CAPABILITIES — AND EVERY STEP LANDING IN EVIDENCE.

> [SCOPE]
>
> This document describes the architecture and live, smoke-tested behavior of the current installation: the agent kernel, the governed capability plane, the durable record, and the deployment forms. Each installation is owned and operated by the organization that deploys it.

2. Capabilities

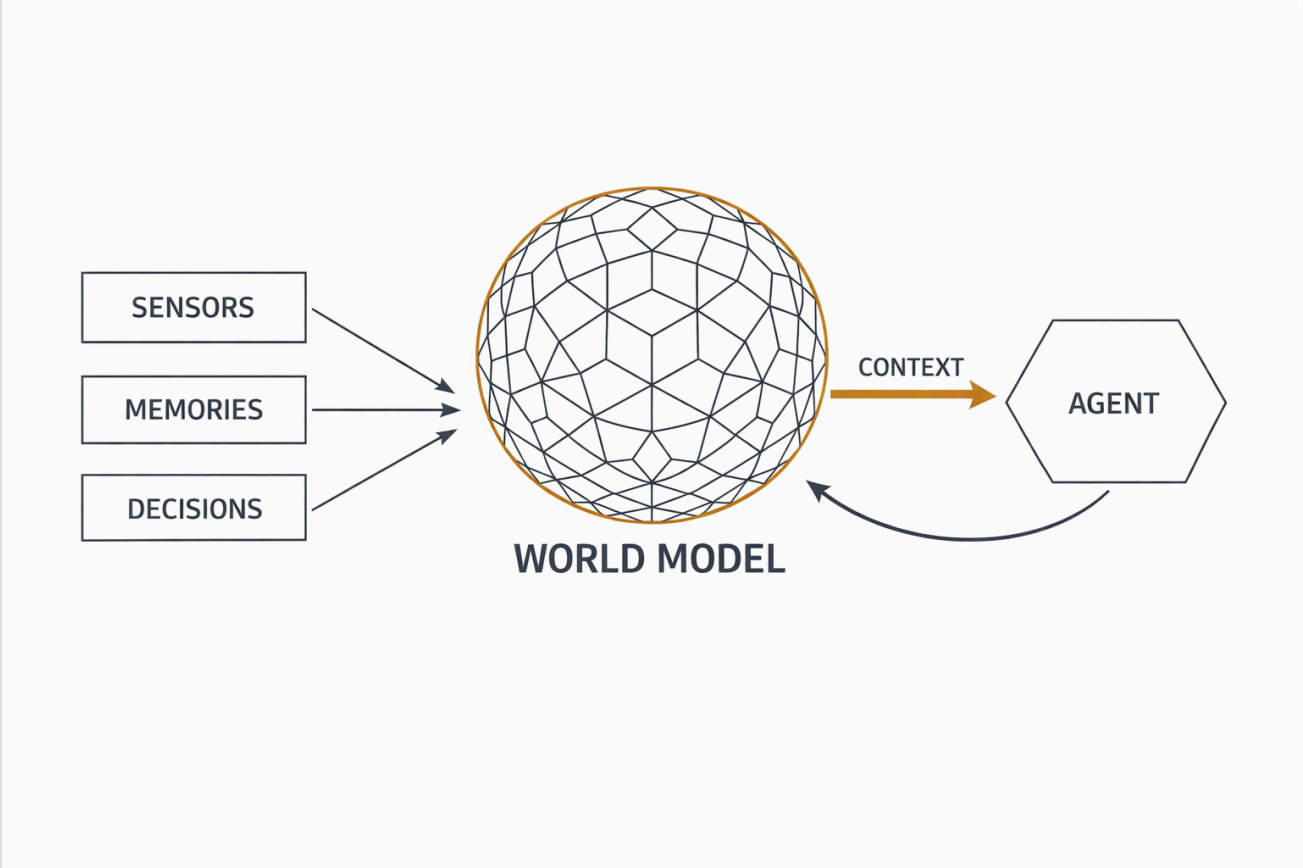


REACTIVE AND PROACTIVE COGNITION: REACT THE MOMENT CONDITIONS CHANGE; DRIVE THE MISSION FORWARD BETWEEN EVENTS — NONSTOP.

- **Detect, decide, intervene — nonstop.** Agents chase the live picture around the clock: they catch conditions the moment they change, decide, and intervene — retiming signals, rerouting vehicles, retasking work — and between events they drive the mission forward on their own initiative. The tempo holds through restarts, upgrades, and machine failures, memory and in-flight work intact.
- **Dispatch long work and move on.** A render, a forecast, a plan, or a 500-step simulation batch carries its own guarantees: it completes on its own, delivers the result the moment it is ready, and an interruption has a defined, recorded outcome.
- **Build a world model of the operation.** Every sensor, memory, and agent decision fuses into one durable world model — ground truth for agents in the moment, an evidence chain after the fact. Any run replays, any moment is queryable, and when someone asks why the system acted, the record answers.
- **Set the rules of engagement.** Every action clears one authenticated door and answers to your policy — who is acting, on which tenant's data, at what sensitivity — refused by default when in doubt. Agents act freely inside the rules, and the rules are yours.
- **Give agents real capabilities.** Generate media, run arbitrary SQL in a bounded sandbox, forecast time series, solve planning problems, profile datasets, transform local frames, route across governed Earth data, render charts, and drive a live viewer — all under one task, artifact, and policy contract. Adding a hosted or remote server is a catalog registration; the platform stays put.
- **Share results deliberately.** Every output is an owned artifact. Grant read, write, or admin to users and groups; mark an artifact releasable and mint a revocable, expiring link. Downloads pass policy and audit before bytes move.
- **Command from one console.** The operations console opens on the running installation: health, tasks, artifacts, agents, recordings, hosted-server topology, policy, audit evidence, and domain administration such as Map sources and releases.
- **Own the whole installation.** Built for contested and disconnected operations — your hostname, your identity provider, your object store — on one host, in a cluster, or fully disconnected.

3. World Model

Agent context is a **world model**: everything the operation senses, remembers, and decides, fused into one durable, queryable model of reality. Sensors and simulators stream the physical world into the record; agents keep their beliefs, missions, and constraints in analytical memory; every decision lands in the log — and each act an agent takes becomes part of the model the next episode reasons from. Context is assembled fresh from the world model every episode, so an agent's knowledge survives any restart and outlives any context window.



SENSORS, MEMORIES, AND DECISIONS FUSE INTO THE WORLD MODEL; EVERY EPISODE'S CONTEXT IS A FRESH SLICE OF IT.

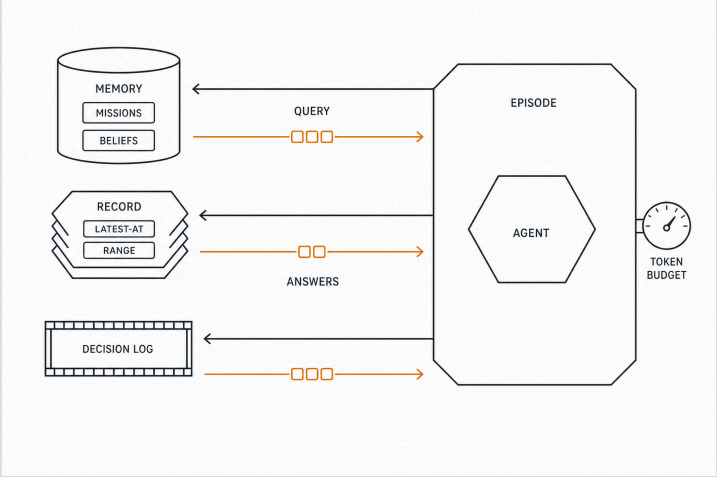
Source	What it contributes	What agents draw
Sensors & sims	The physical world, streamed into the durable time-and-space record.	The world now (latest-at) · how it got here (trajectories and ranges).
Agent memory	Beliefs, missions, constraints, and state in each agent's analytical memory.	What the agent believes and intends — queryable with guarded SQL.
Decision log	Every turn, tool call, and rationale, time-indexed and replayable.	What happened and why — the model of the agents themselves.

3.1 Context, Queried

The world model changes what an episode's context is made of. Context here is assembled by query: the kernel composes each episode from manifest-defined slices, and the agent holds the same instruments, so mid-episode it asks for exactly what it needs — rows from its own memory, a snapshot or trajectory from the record, analytics over anything it has ingested.

Instrument	Example	What the episode gets
memory_query	`SELECT id, eta, risk FROM missions WHERE status = 'active'`	Exactly the active missions, as rows; beliefs and constraints join the same way.
query_recording · latest-at	every vehicle's position, now	A world snapshot, measured — the state of reality in one call.
query_recording · range	vehicle 214, last five minutes	One trajectory, on the original timeline.
timeline_query	my tool calls and outcomes, last six hours	The agent reads its own conduct as data.
duckdb__query	`SELECT class, count(*) FROM detections GROUP BY window_5m`	Analytics over anything ingested — including perception output.

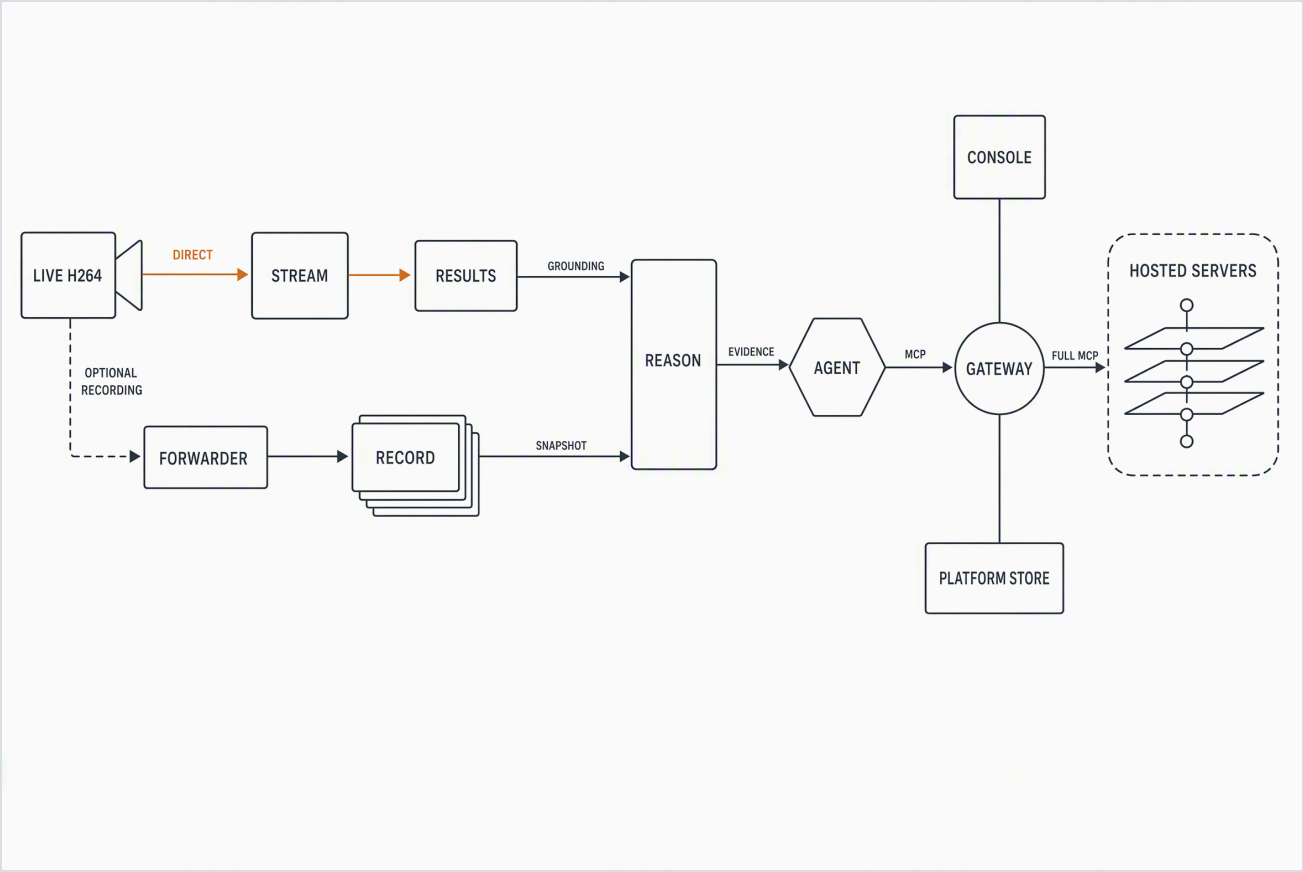
The strategy carries for two reasons. Tokens spend on answers rather than history, so a mission's entire history compresses to the rows this episode needs. And because every instrument reads the same governed planes, results join — detections land in SQL, SQL rows feed planning, committed plans write back to memory.



THE EPISODE QUERIES THE PLANES — MEMORY, RECORD, DECISION LOG — AND ONLY ANSWERS COME BACK TO SPEND TOKENS, UNDER A HARD BUDGET.

4. Data Flow

The decision cycle, engineered: Boyd's OODA loop, with orientation living durably in the world model — and one stage Boyd never needed, Account. **Observe**: live H.264 enters Stream directly while an independent forwarder may publish governed recording evidence; agents query immutable snapshots back out through Recording and Reason. **Act**: agents and operators call tools through the gateway; long work becomes a durable task whose completion wakes the sleeping agent. **Account**: every auth decision, tool call, task transition, artifact grant, and agent step lands in the platform store and the decision log.

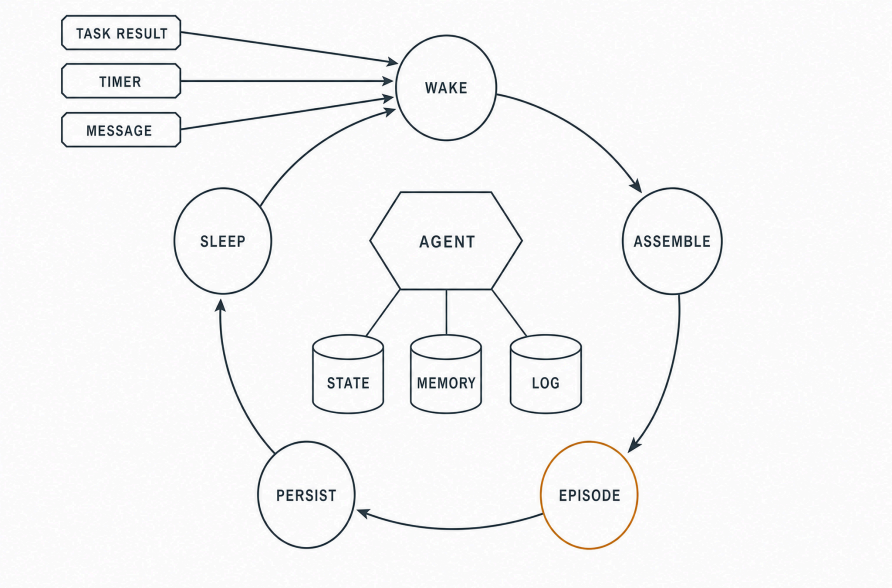


THE WHOLE SYSTEM: LIVE STREAM NEVER WAITS FOR RECORDING HUB; STREAM RESULTS AND AUTHORIZED RECORDING SNAPSHOTS GROUND REASON BEFORE EVIDENCE REACHES THE AGENT.

Flow	Path	Guarantee
Observe	Arriving H.264 enters Stream directly; a local forwarder uploads recording batches; Stream results and authorized snapshots feed Reason.	Live results never wait for Recording Hub; acknowledgement follows fsynced journal state and part materialization.
Act	Every tool call enters through the gateway under one flat namespace; quick tools answer inline; long tools return a durable task handle immediately.	Task completion pushes a wake; the next episode assembles with the result.
Account	Auth, policy, admin, and task transitions write audit evidence; every decision records what the agent saw, chose, and why.	Usage records meter provider spend and tool activity per principal.

5. Agent Kernel

An agent runs forever because durable stores are the memory: every episode's context is assembled fresh from the agent's memory planes, and everything in flight survives process death. The kernel validates every manifest against its schema before execution. An agent type combines a manifest, a preamble, and SQL migrations.



WAKE → ASSEMBLE → EPISODE → PERSIST → SLEEP, FOREVER.

Stage	What happens
Wake	A batch of wakes, coalesced and deduped; every wake is persisted with its disposition, so the ledger explains why each episode ran.
Assemble	State header, wake payload, and manifest-defined SQL sections over the memory database, under a token budget.
Episode	One bounded LLM run. Quick gateway tools answer inline; long tools dispatch as durable tasks. Hooks record every step and terminate over-budget runs.
Persist	Unresolved task descriptors land in the platform store; the episode closes with usage counters and a summary.
Sleep	Idle on the wake bus. A leased watcher per detached task waits on the result; the heartbeat bounds silence.

Wakes: task result · resource change · timer · operator message · elicitation answer. A wake survives process death — descriptors and watcher leases rehydrate from the platform store on any boot, so kill and restart resumes the same tasks.

5.1 Memory Planes and Durability

Memory planes

- **Scheduling truth.** The SurrealDB platform store owns episodes, wakes, task watchers, and leases; the outbox and changefeed deliver wakes across processes.
- **Analytical memory.** A sandboxed DuckDB per agent: domain tables (the Pilot: `targets`, `missions`, `waypoints`, `constraints`, `beliefs`), a kv store, and an episode-log projection for context assembly.
- **Decision log.** An append-only Rerun recording: every turn, tool call, task lifecycle, prompt, and summary — time-indexed and watchable live.
- The agent reads all three itself: `memory_query` (guarded SQL), `memory_write` (bounded mutations), and `timeline_query` (dataframe reads over the log).

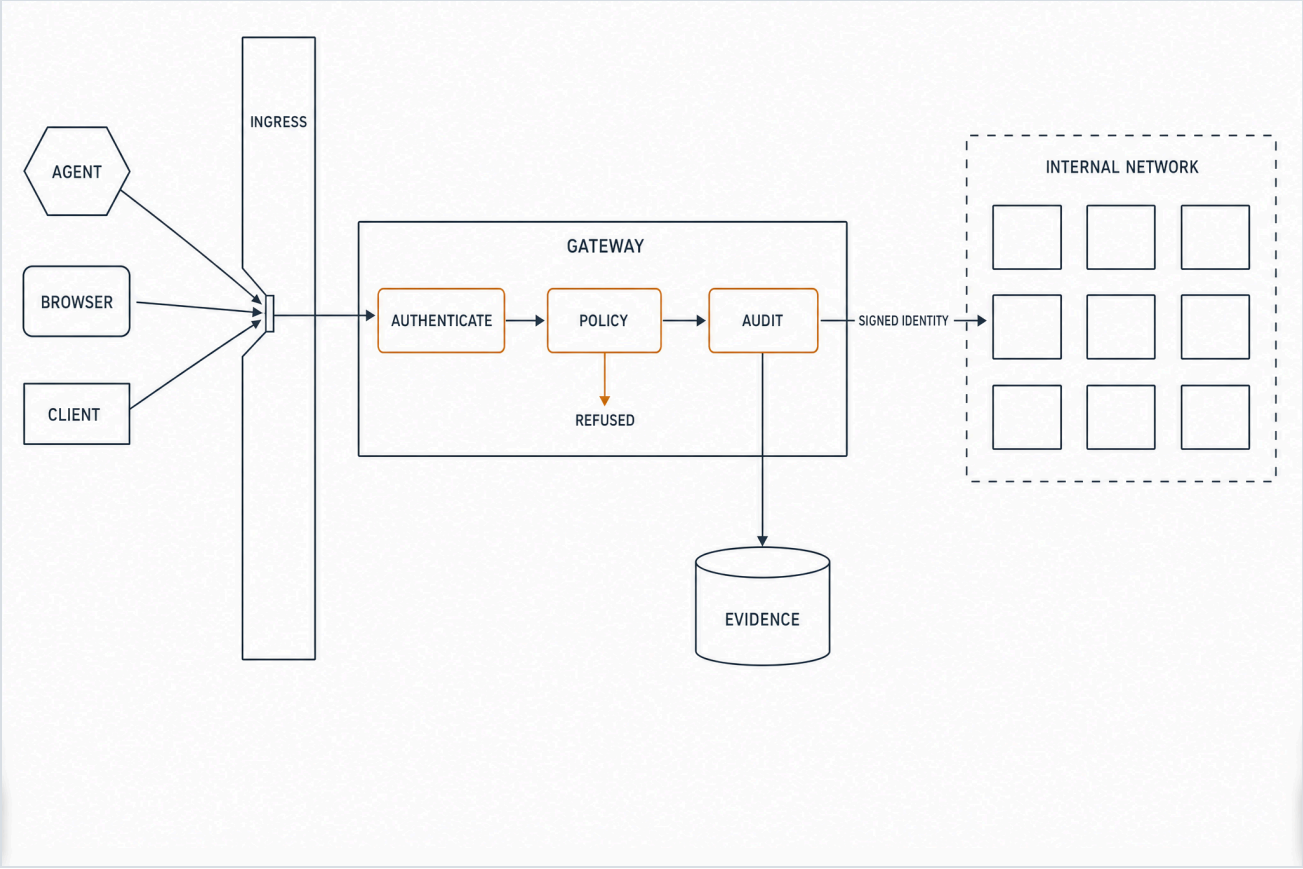
Durability, proven

- Detach at episode end: a task's descriptor outlives the process; any later boot rehydrates the live handle and keeps waiting.
- Token rotation is make-before-break; task ids are principal-scoped, so continuity holds across reconnects.
- The wake is push, arithmetically: with polling pinned at 60s and the heartbeat at 600s, the smoke's agent slept the task's full ~15s and woke within a second of completion — verified live with a real model end to end.
- Budget hooks book overruns as `budget_terminated`, a first-class outcome; hourly windows defer non-priority wakes.

> [PRINCIPLE]
>
> Agents are data. At boot, schema validation rejects an invalid manifest. The manifest declares the model endpoint, gateway principal, episode limits, budgets, schedule, context SQL sections, and writable tables. A preamble and domain migrations complete the profile; cross-episode instructions live there or in memory.

6. Edge, Access, and the Gateway

Every client enters through the installation's own Kubernetes Ingress. `/mcp`, `/oauth`, `/admin`, and `/artifacts` reach the gateway; `/console` and `/auth` reach the console BFF; `/s/{token}` reaches the artifact service for public links. Every hosted server lives on the internal network and requires a gateway-signed Ed25519 identity assertion.

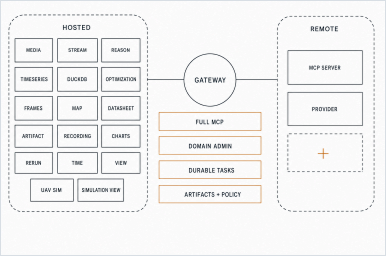


EVERY ACTION RUNS THE GAUNTLET: ONE DOOR IN, AUTHENTICATE → POLICY → AUDIT, REFUSED BY DEFAULT, SIGNED IDENTITY ONWARD, EVIDENCE KEPT.

Function	Behavior
Authentication	Browser OAuth with PKCE, client credentials with private-key JWT, identity-assertion grant (ID-JAG) exchange, JWKS, token issuance and revocation, durable refresh rotation — reuse revokes the family.
Policy & audit	Attribute- and role-based policy over tools, resources, tasks, and artifacts; tenant and data-label checks for regulated data; fail-closed decisions with reason codes; a durable audit trail for admin, auth, and policy operations.
Control plane	Validated server / profile / policy catalog, authenticated live apply and reload, secret references with redaction, an admin API for the console, and policy-checked, audited artifact downloads.
Domain administration	Scoped MCP tools, resources, durable tasks, and Apps are canonical. A declared `/admin/{profile}/servers/{server}/{path}` route is an additive projection over the same typed models, policy, audit, identities, and state.
Protocol surface	Tools, resources and templates, prompts, completions, tasks, subscriptions, notifications, structured content with declared schemas, artifacts, and usage. Tool names gain a prefix only at aggregation; resource URIs retain their owning scheme.

7. Hosted Capabilities

The canonical control plane defines seventeen hosted server identities. Each server declares the MCP surfaces its domain needs and runs behind gateway-signed identity; a new local, bridged, or **remote MCP endpoint** registers in the validated catalog and immediately answers to the same task, artifact, policy, and audit contracts. The gateway discovers the declared surface, prefixes tool names only at aggregation, and preserves each server's resource URI scheme.



SEVENTEEN HOSTED SERVERS AND REMOTE ENDPOINTS — FULL MCP, CANONICAL DOMAIN ADMINISTRATION, DURABLE TASKS, ARTIFACTS, AND POLICY REMAIN ONE GOVERNED SYSTEM.

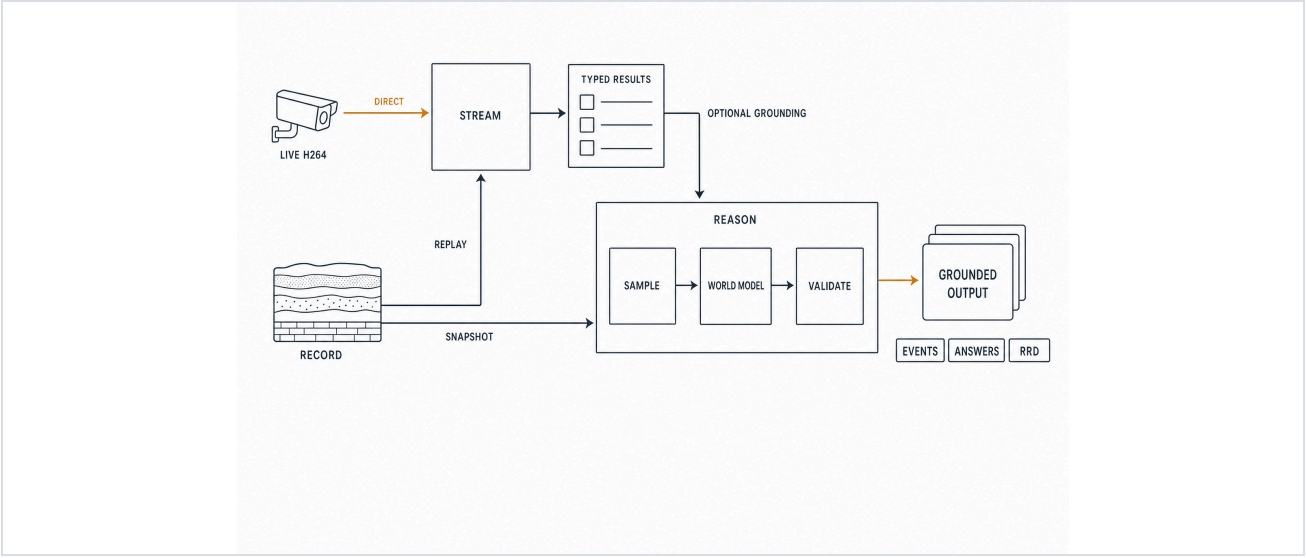
Server	Capability	Representative tools
Media `media`	Provider-backed generative media with webhook-driven completion, a model catalog, and pricing.	`run · models · model_schema · artifact`
Stream `stream`	Direct live RTP/H.264 processing plus reproducible replay over authorized task-start snapshots, with encoded App preview and typed results.	`start_live_session · stop_live_session · run_recording`
Reason `reason`	Local world-model reasoning over recording snapshots, optionally grounded by governed Stream perception results.	`analyze_recording`
Forecasting `timeseries`	Forecasting over any SQL-readable time-series source, visualized as Rerun recordings.	`forecast`
Time `time`	Authority-bound civil, military, GNSS, and mission time; calendars, timelines, clock quality, and temporal events.	`resolve_time · convert_time · assess_clock · evaluate_windows`
SQL `duckdb`	Arbitrary SQL over principal-owned databases inside a locked, resource-bounded sandbox.	`query · execute · ingest · export`
Planning `optimization`	Task-option planning for one or many agents over planning objects or SQL-readable option rows.	`plan`
Frames `frames`	Complete ECEF-rooted frame worlds, immutable revisions, bounded conversion, durable batches, and provenance.	`create_world · publish_world · convert_frame · batch_transform`
Map `map`	Earth geography, governed releases and restrictions, Valhalla and governed-network routes, matrices, and reachable areas with pinned provenance.	`search_locations · inspect_location · route · route_matrix · reachable_area · publish_restriction · validate_route`
Datasheet `datasheet`	Python reference server for dataset preview, column statistics, and durable artifact-backed profiling.	`preview_dataset · column_stats · profile_dataset`
Sharing `artifact`	Discovery, metadata, grants, release, and sharing — the governance surface over the byte store.	`metadata · grant_access · revoke_access · set_release_state · create_share_link · revoke_share_link`
Recordings `recording`	Authorized access to the Recording Hub: discovery, dataframe queries, subscriptions, publication.	`query_recording · seal_recording`
Charts `charts`	Chart rendering projected through the gateway, turning query results into shareable visuals.	`render_chart · compile_chart · validate_chart · list_chart_types · create_chart_view`
Viewer `rerun`	A live Rerun viewer driven entirely over MCP — session, capture, inspection, and UI automation.	`17 interactive tools (connect … batch)`
3D View `view`	Governed static scene compositions with exact inputs and bounded overlays, rendered offscreen on NVIDIA Vulkan.	`create_scene_composition · create_view · set_camera · capture_frame · close_view`
UAV Simulation `uav-sim`	Governed sessions, missions, vehicles, terrain, dataset capture, and provider-neutral simulator control.	`configure_world · execute_mission · capture_dataset`
Simulation View `simulation-view`	Renderer-independent scenes, logical cameras, capacity, live-view leases, WebRTC, and a GPU-backed MCP App.	`create_session · create_camera · open_live_view`

7.1 In Depth: Live Stream, Replay, and Grounded Reason

Stream begins processing arriving RTP/H.264 frames immediately. Its encoded tee feeds a bounded App preview and may feed the independent recording forwarder, but live results never wait for Recording Hub. The same admitted pass-through or perception profile can run later over one immutable task-start snapshot of complete acknowledged recording parts.

- **Live:** start one owner-scoped session, process new frames on the GPU, and publish typed detections while the H.264 App draws overlays.
- **Replay:** run the same approved profile over an authorized recording range and publish immutable JSON and RRD results.
- **Reason:** combine an authorized recording snapshot with a bounded typed subset from `stream://artifact/{id}`; answers may cite Stream track identities.

Stream processing, Reason inference, and visual acceptance fail closed without their declared hardware. Browser H.264 software decode is allowed only when Media Capabilities reports the exact configuration as supported and smooth, and the App labels that path as software.



LIVE STREAM AND AUTHORIZED REPLAY PRODUCE GOVERNED RESULTS; REASON COMBINES AN IMMUTABLE RECORDING SNAPSHOT WITH OPTIONAL TYPED STREAM GROUNDING.

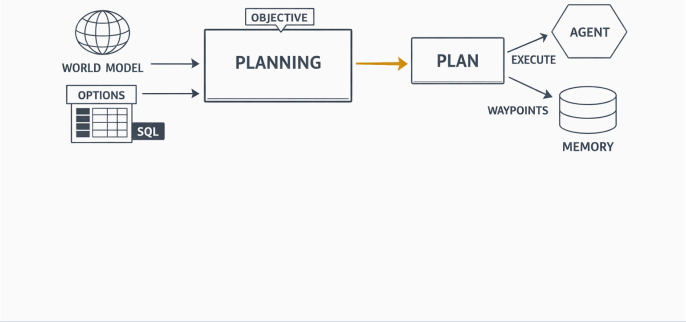
7.2 In Depth: Planning

Planning solves assignment: given agents or assets, the options open to them, and an objective, it returns the best allocation — who does what, in what order, at what cost. The kinds of questions it answers:

- Three survey drones, forty waypoints, battery limits — which drone flies which route?
- Given measured demand per approach, which signal-timing plan minimizes total delay?
- Ten targets, two sensors, shifting priorities — what gets watched this hour?

Options are rows: any SQL-readable table is a valid option set — the mission tables an agent already keeps, or the demand counts perception produced minutes ago. The result is a structured plan the agent executes tool by tool, plus a visualization an operator can inspect before it commits; committed waypoints land back in agent memory.

The request compiles to an exact optimization: one binary decision per option, a linear objective weighing priority, confidence, cost, risk, duration, and resource use under operator-set weights, and constraints covering task requirements, agent concurrency, resource ceilings, dependencies, exclusions, and time-window conflicts between options that share an agent. An embedded solver settles the model in-process, which is why the identical planner runs on an edge host or behind an air gap. The answer is optimal under the declared objective or explicitly infeasible, and it returns with the scores and constraint counts that decided it — the same inputs produce the same plan, and the plan can explain itself.



THE WORLD MODEL AND SQL-READABLE OPTIONS GO IN; THE PLAN COMES OUT — EXECUTED BY THE AGENT, WRITTEN BACK TO MEMORY.

One governed loop

#	Call	Result
1	`wake — resources/updated: sumo://congestion`	A jam forming is a wake, the moment it happens.
2	`query_recording · latest-at`	World snapshot: edge E14 saturated.
3	`stream__run_recording /world/camera/e14 · 10 min`	1,204 detections · 87 tracks — cars, buses, trucks, counted per approach.
4	`duckdb__ingest → query`	Demand per approach, as SQL rows.
5	`optimization__plan · minimize total delay · options from SQL`	The winning signal-timing plan, as structured data.
6	`sumo__set_signal_phase · run_batch(500)`	Applied; the agent sleeps through the batch.
7	`wake — task result`	Delay –23%; every step already in the record.

8. Durable Tasks and Shared Planes

Shared durable tasks

Every long-running tool runs on the shared task runtime and the final Veoveo task extension. Task IDs are UUIDv7; creation, leases, transitions, results, and outbox events are atomic in the platform store. Recovery is declared per operation:

Class	Meaning
`resume`	Deterministic, side-effect-safe work may be reclaimed after its lease expires.
`webhook_wait`	A durably submitted provider job waits for its signed webhook. Completion is webhook-only.
`interrupted_indeterminate`	Interrupted mutating work fails closed and is never replayed automatically.

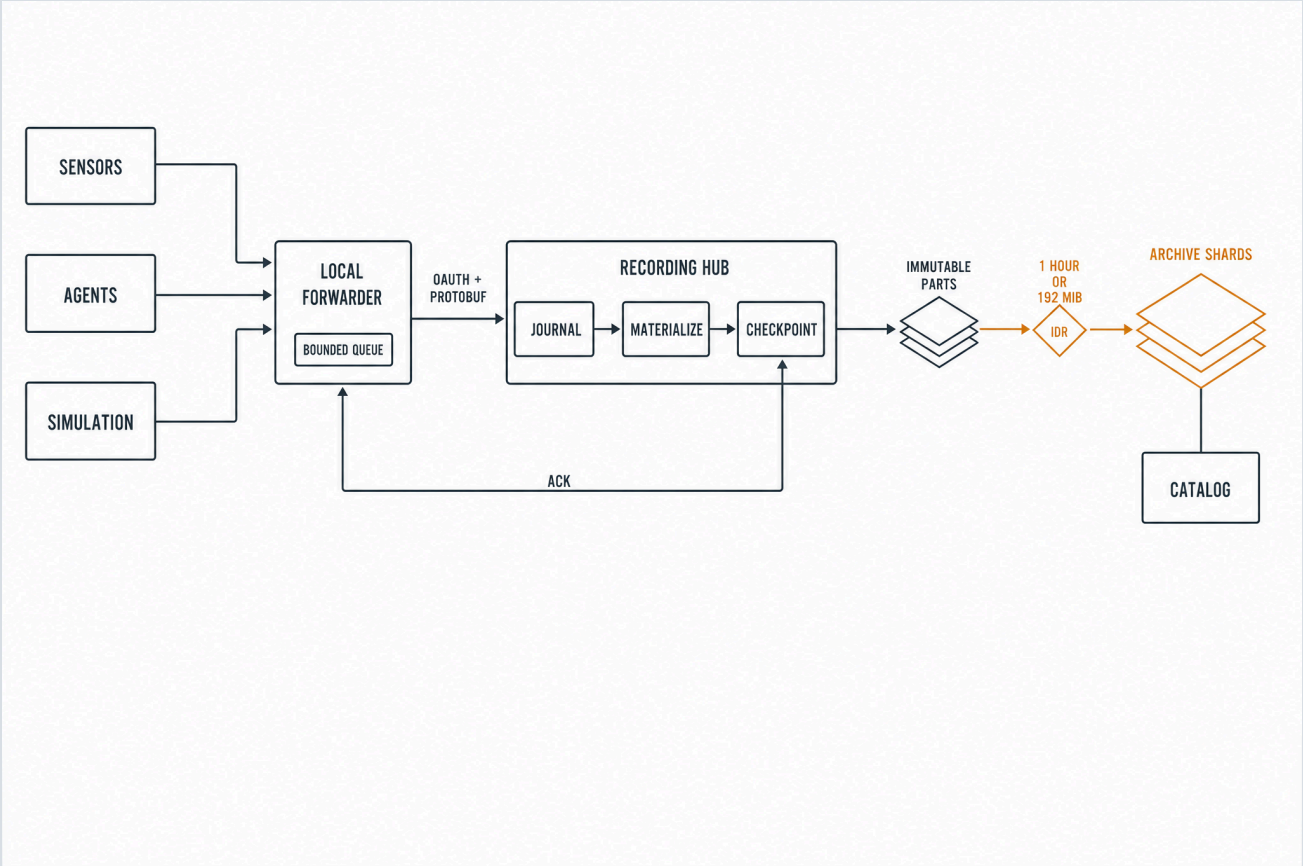
Shared planes

- **Platform store.** One SurrealDB 3.2.1 node (RocksDB) is the durable coordination authority: identity, policy, control-plane revisions, tasks, artifacts, recordings, map catalogs and releases, agents, wakes, audit evidence, and usage. The transactional outbox and replayable changefeed carry all cross-process work.
- **Artifact plane.** A single byte-level policy-enforcement point. S3-compatible object store for bytes; the platform store owns grants, labels, and release state. Fresh opaque identity per occurrence (`artifact://{uuidv7}`); hashes provide integrity and tenant-local dedup. `artifact://` references let one server consume another's output.
- **Contract core.** The provider-neutral contract every server shares: Rust newtypes and enums, deployment models, exported JSON Schemas, and gateway-to-server Ed25519 identity assertions — all validated fail-closed; servers hold public verification keys only.

> [PRINCIPLE]
>
> Two sharing modes, both explicit: authorized grants to users or groups, still constrained by tenant and label policy; and read-only anyone-with-link bearers for artifacts explicitly marked releasable — expiring, download-limited, and revocable.

9. Recording Hub

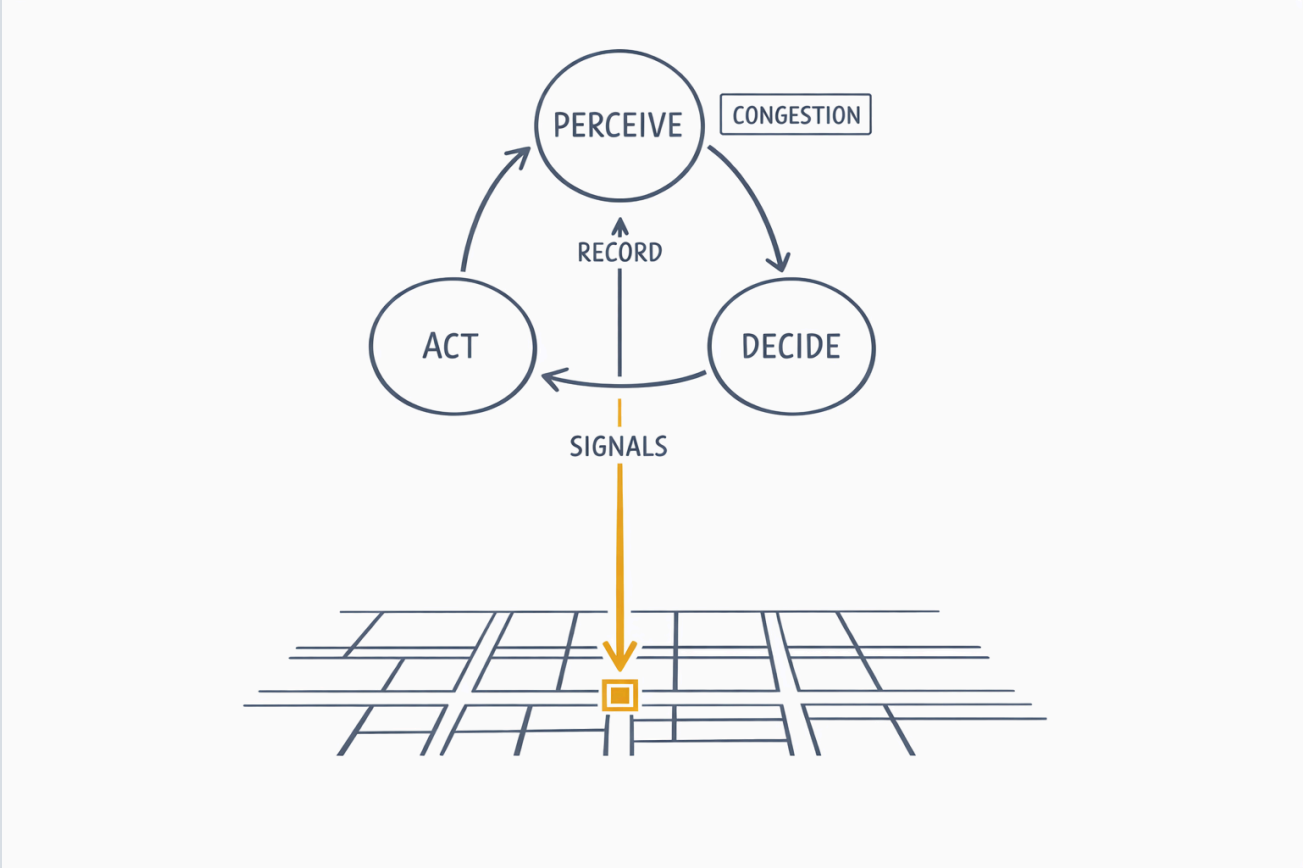
Each producer publishes native Rerun traffic to a loopback forwarder with a bounded persistent queue. The forwarder authenticates through the gateway and resumes versioned protobuf upload from the Hub checkpoint. Recording Hub fsyncs journal state and materializes an immutable ordered RRD part before acknowledgement. One cataloged writing segment closes at one hour or the 192 MiB precompaction safety limit, aligned to a decoder-reentrant IDR. Archive publication is one verified footer-writing object-store pass.



DURABLE CAPTURE: FSYNCED JOURNAL → CHECKPOINT → IMMUTABLE PARTS → INDEXED SHARDS. A REPRESENTATIVE 30,307-CHUNK CORPUS BECAME 31 ARCHIVE CHUNKS IN 2.56 SECONDS.

10. SUMO Showcase

SUMO — Simulation of Urban Mobility, an open-source traffic microsimulator — runs the pinned, MIT-licensed LuST Luxembourg scenario in its own container. One `sumo-mcp` server owns the single serialized TraCI connection, pushes the world into the hub as Rerun `/world/sumo/**` streams, and exposes SUMO control as governed `sumo__*` tools. The long operations are durable tasks the agent detaches from and wakes on — the same sleep/wake the kernel already runs, now driven by a real simulator.

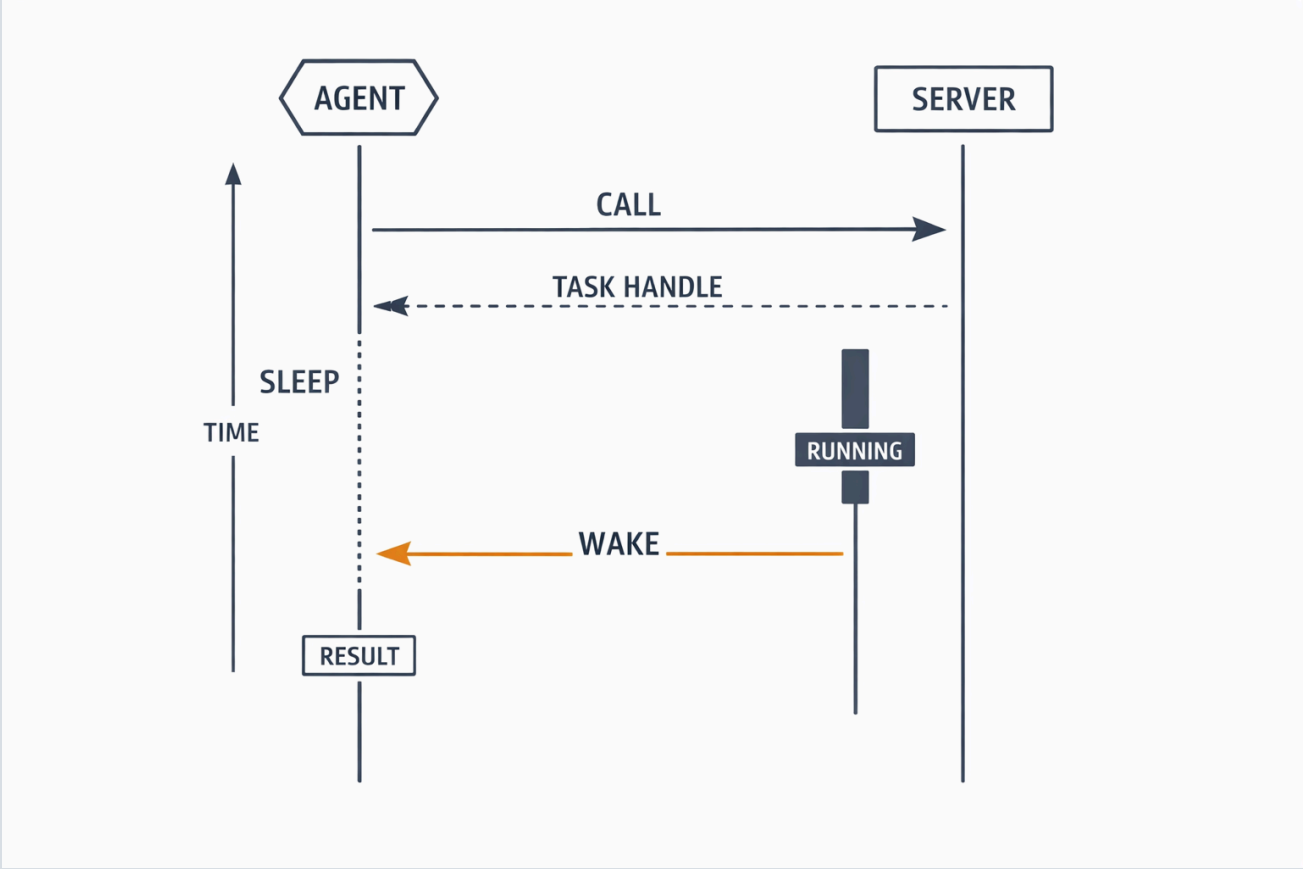


A LIVE TRAFFIC WORLD PROVING PERCEIVE → DECIDE → ACT, END TO END.

Surface	Detail
Sync tools	`query_state` · `describe_scenario` · `set_signal_phase` · `reroute_vehicle` · `set_edge_speed` · `close_lane` · `open_lane`
Durable tasks	`run_batch` (live sim — `interrupted_indeterminate`) · `generate_network` · `compute_routes` · `optimize_signals` (real SUMO CLIs, resumable)
Events	`sumo://congestion` is subscribable — a jam forming is a wake the moment it happens.
Outputs	Successful task outputs upload through a task-bound write capability to the shared artifact plane.

10.1 Task Sleep/Wake

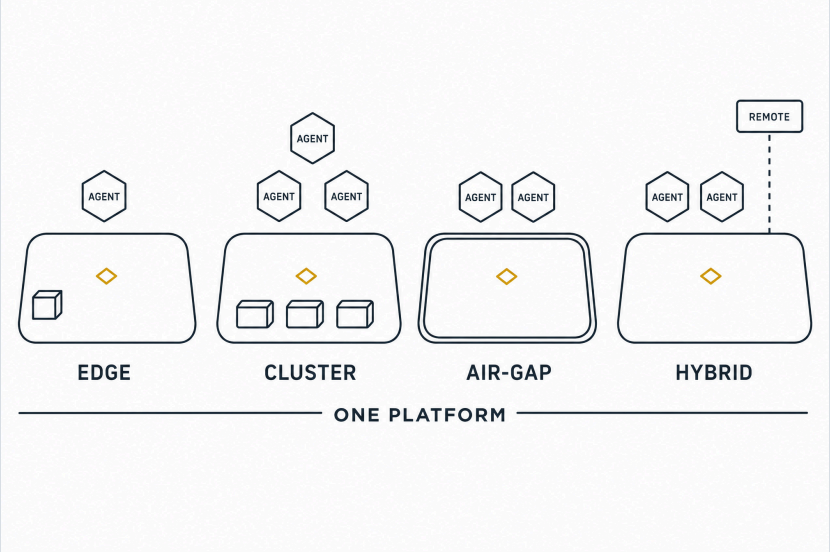
The exact detach → sleep → wake path, on the final task extension the gateway and kernel speak: the agent calls a task-augmented tool, receives a durable handle immediately, persists the descriptor, ends the episode, and sleeps. The server runs on; completion pushes a wake, and a fresh episode assembles with the result — within a second, across process death.



DETACH → SLEEP → WAKE: ZERO CONNECTIONS HELD, ZERO EPISODES BURNED WAITING.

11. Operations and Deployment

Deployment is a spectrum, and cognition is constant across it. The same platform runs at the operation's edge on one host, across a cluster, sealed inside an air gap, or hybrid — an edge installation that reaches remote capability when links allow. Wherever it lands, it carries the full stack: **one agent or a team of them, every hosted capability server, and the operational intelligence to run the mission** — the console, the record, and the audit trail.



SAME PLATFORM, SAME COGNITION — EDGE, CLUSTER, AIR-GAP, OR HYBRID; ONLY HYBRID REACHES OUT, AND ON YOUR TERMS.

Form	Shape
Local	`cargo xtask smoke profile-cluster-up --profile <profile>` creates a loopback-only Kubernetes cluster. The same typed profile publishes immutable images and installs the workload model used in the field.
Cluster	The Helm form: one SurrealDB StatefulSet, default-deny NetworkPolicy, optional service-mesh mTLS, separate migration and runtime credentials, operator-supplied telemetry and SIEM wiring.
Air-gap	The verified offline bundle contains pinned images, Helm material, config schemas, checksums, and SBOMs. Installation and verification complete entirely inside the boundary.
Hybrid	An edge installation that registers remote MCP endpoints and provider-backed servers. The reach is governed like everything else — same policy, same audit, same task contract.

Operational intelligence, wherever it lands

- **Command from the console.** The first screen is the live installation — health, tasks, artifacts, agents, recordings, policy, and audit. Map sources, acquisitions, releases, active pointers, and mobility profiles are managed through canonical MCP tools, resources, durable tasks, and the Map administration App.
- **Cognition scales in place.** Agents are data: field another agent by adding a manifest. The same wake bus, budgets, and memory planes run one agent or a team.
- **Evidence deploys with the platform.** The record, the audit trail, and usage metering are part of every form — air-gapped included.

12. Verification and Observability

- **Conformance CLI.** A generic MCP client exercising the full protocol surface — info, resources, prompts, completions, runs, usage, artifacts — plus auth-metadata discovery and token tooling.
- **Smoke harness.** End-to-end scenarios with real process orchestration: fake IdP, provider, and upstream services; a real Keycloak realm; live agent-kernel sleep/wake; and the full SUMO world in a disposable k3d cluster. The Map smoke acquires and activates source fixtures, runs real Valhalla and governed-network routes through tasks, and proves restrictions and release changes invalidate dependent resources.
- **CI.** Formatting, clippy, Rust and Python tests, the datasheet template, the console UI build, the Keycloak integration path, and deployment contract checks run on every change.
- **Hardware visual acceptance.** Headed browser checks probe WebGPU and WebGL and require at least one hardware-backed path. SwiftShader, llvmpipe, and software rasterizers are rejected. GPU workloads request their accelerator and have no CPU fallback.
- **Telemetry.** OpenTelemetry traces and logs across gateway and servers, with audit summaries as durable evidence for regulated deployments.

> [EVIDENCE]
>
> Every capability in this document is live, smoke-tested behavior of the current installation. The record replays, the audit trail is durable, and the decision log carries each agent's rationale — when someone asks why the system acted, the record answers.

13. Component Reference

Component	Role	What it is
ingress	ours	The single public door, owned by the operator: Kubernetes Ingress in local k3d and fielded clusters. One place to defend and observe.
gateway	ours	The governed protocol boundary: authenticates the caller, checks policy, projects full MCP server surfaces, carries declared additive administration projections, and writes the audit trail.
console	ours	The cockpit: the operations UI plus a session backend that owns browser login, encrypted sessions, and CSRF enforcement.
agent kernel	ours	The runtime for long-lived agents: short episodes, durable memory, wake on a result, a timer, or a message.
hosted MCP server	ours	A domain boundary with typed models, full MCP surfaces, shared task and artifact contracts, canonical MCP administration, and only declared additive HTTP projections.
Map server	ours	Earth geography and logistics routing over governed releases; scoped MCP tools, resources, tasks, and Apps own sources, acquisition, activation, rollback, and mobility profiles.
world model	concept	What an agent knows: everything the operation senses, remembers, and decides, fused into one durable, queryable model of reality. Every episode's context is a fresh slice of it.
platform store	OSS	SurrealDB 3.2.1 — the platform's memory: identity, policy, tasks, artifacts, recordings, map catalogs and releases, agents, audit, and the transactional outbox.
task runtime	ours	The ledger for long work: UUIDv7 task ids with atomic leases, transitions, results, and declared recovery per operation.
recording forwarder	ours	The producer-local durability edge: loopback Rerun gRPC, bounded persistent queue, OAuth upload, and checkpoint resume.
Recording Hub	ours	The acknowledgement authority: fsynced batch journal, immutable ordered part materialization, and a monotonic checkpoint.
RRD archive shard	format	An immutable footer-indexed shard produced once at an IDR-aligned hour or 192 MiB boundary.
catalog	ours	The card catalog: stable recording identities related to acknowledged parts, archive shards, checkpoints, labels, and lifecycle.
recording server	ours	The reading room: authorized discovery, latest-at / range / dataframe queries, subscriptions, and sealing to an artifact.
SUMO / TraCI	OSS	The traffic simulator and its control socket — a live world producing positions and states to record and reason about.
sumo-mcp	ours	The bridge: the one owner of the simulator's control socket, pushing the world into the hub and exposing governed control.
MCP	protocol	The shared protocol for tools, resources, prompts, completions, tasks, subscriptions, notifications, and structured content with declared schemas; one gateway governs the complete surface.
durable task	concept	A tool call you walk away from: the handle returns immediately; the row, lease, and result live in the platform store.
resource · subscribe	concept	A condition that wakes you: subscribing to a resource means the agent is notified the moment it changes.